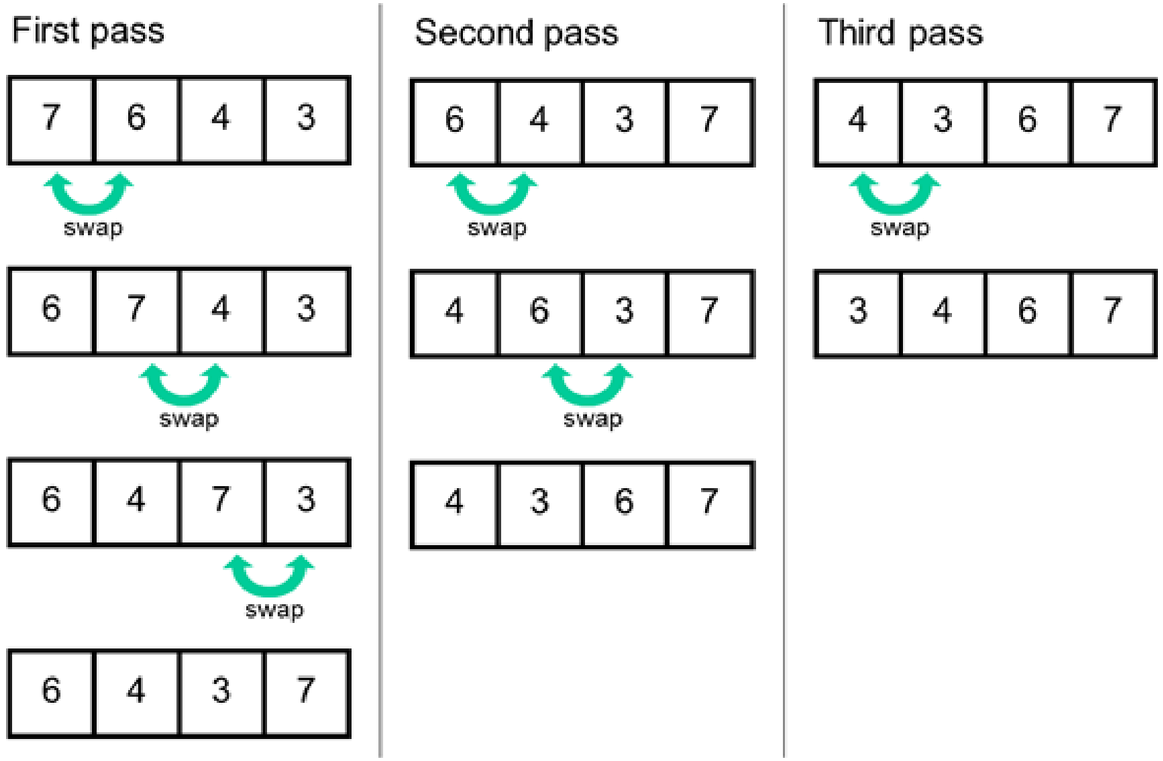
進階程式設計
Advanced Program Design

L4

Chia-Chi Chang
ccchang@cc.ncue.edu.tw

氣泡排序法(Bubble Sort)

- 若要進行由小排至大，須將相鄰兩個數值進行比較，將「數字大」交換至右邊，如下圖所示；反之，若由大排至小，則須將「數字小」交換至右邊。
  - 若有n個資料要排序，通常需要n-1輪掃描；每進行完一輪排序數目會遞減1
    (※每一輪的最右邊的數值為該輪掃描中的最大者)。
- 共執行(n-1)+(n-2)+…+2+1=n(n-1)/2次，時間複雜度$O(n^2)$
  - 最糟情況(Worst Case)：$O(n^2)$
  - 最好情況(Best Case)：O(n)「至少要比較n次」。



Q3: 成績排序

Q2: 撰寫一程式讀取10筆分數(0~100)，然後按照成績由低至高排列。

範例1輸入：65 80 37 55 42 99 55 60 60 45

範例1輸出：37 42 45 55 55 60 60 65 80 99

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
void bubble_sort(uint16_t []);
int main(int argc, char *argv[]) {
    uint16_t loop, grade[10] = {0};

    for(loop=0; loop<10; loop++) {
        scanf("%hu", &grade[loop]);
    }
    bubble_sort(grade);
    return 0;
}
```

```c
void displayArr(uint16_t grade[10]) {
    for(uint16_t loop=0; loop<10; loop++) {
        printf("%d ",grade[loop]);
    }
    printf("\n");
}
void bubble_sort(uint16_t grade[10]) {
    uint16_t iloop, jloop, t;

    for(iloop=0; iloop<9; iloop++) {
        for(jloop=0; jloop<9-iloop; jloop++) {
            if(grade[jloop] > grade[jloop+1]) {
                t = grade[jloop];
                grade[jloop] = grade[jloop+1];
                grade[jloop+1] = t;
            }
        }
    }
    displayArr(grade);
}
```

關於break的觀點

數字反轉(佔分 25 分)

問題描述：

試撰寫程式將其輸入的數字反轉後輸出。

輸入說明：

讀取 k 個整數 m，直到讀取-1 結束，其中 1≤k≤100，0≤m≤1000000000。

輸出說明：

依序輸出反轉後的數字。

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main(int argc, char *argv[]) {
    int32_t m;

    while(1) {
        scanf("%d", &m);
        if(m == -1) {
            break;
        }
        printf("m = %d\n",m);
    }
    return 0;
}
```

使用break離開無窮迴圈

軟體工程的觀點 4.3

某些程式設計師覺得 **break** 和 **continue** 違反結構化程式設計的規範。我們很快就會討論到，因爲這些敘述式的效果可以由結構化程式設計技術取代，所以這些程式設計師不使用 **break** 和 **continue**。

增進效能的小技巧 4.1

當正確地使用 **break** 和 **continue** 敘述式時，其執行速度會比相應功能的結構化技巧來得快（我們很快會學到這些技巧）。

軟體工程的觀點 4.4

在獲取良好軟體工程品質和最佳執行效率之間，常存在著矛盾。通常是魚與熊掌不可兼得。在大多數要求效率的情況下，請遵守以下原則：先求程式碼簡單正確，若有需要再求小巧快速。

避免使用break去跳離迴圈

數字反轉(佔分 25 分)

問題描述：

試撰寫程式將其輸入的數字反轉後輸出。

輸入說明：

讀取 k 個整數 m，直到讀取-1 結束，其中 1≤k≤100，0≤m≤1000000000。

輸出說明：

依序輸出反轉後的數字。

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main(int argc, char *argv[]) {
    int32_t m;

    while(1) {
        scanf("%d", &m);
        if(m == -1) {
            break;
        }
        printf("m = %d\n",m);
    }
    return 0;
}
```

使用break離開無窮迴圈

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
int main(int argc, char *argv[]) {
    int32_t m;

    while((scanf("%d", &m) != EOF) && (m != -1)) {
        printf("m = %d\n",m);
    }
    return 0;
}
```

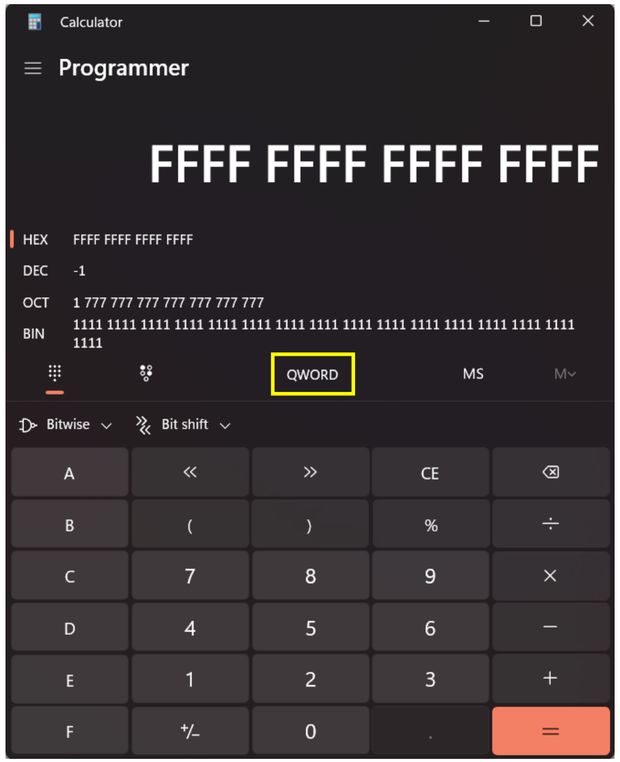
單一入口/單一出口的循環敘述式使得程式變得簡單且容易理解，有助於除錯

Number System

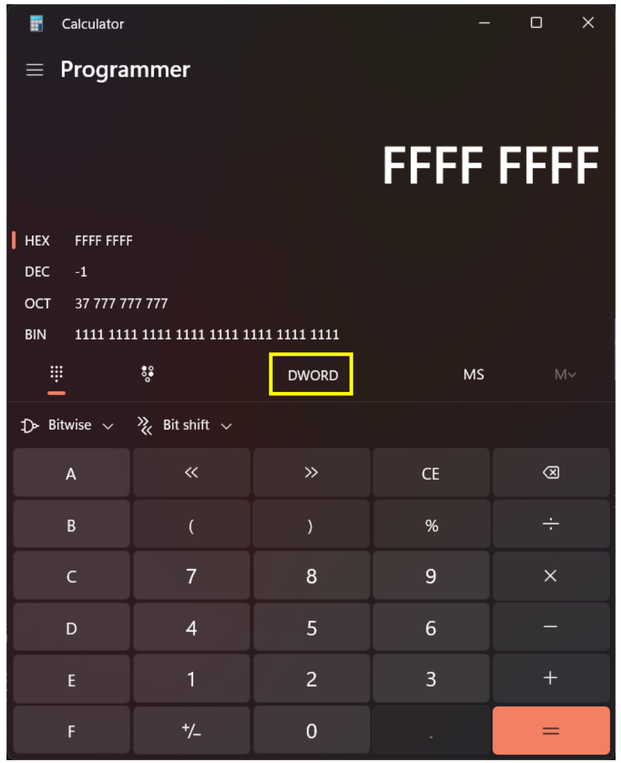
Decimal (10進制)	Binary (2進制)					Octal (8進制)		Hexadecimal (16進制)	
	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Bit 1	Bit 0	Bit 1	Bit 0
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	1	0	1
2	0	0	0	1	0	0	2	0	2
3	0	0	0	1	1	0	3	0	3
4	0	0	1	0	0	0	4	0	4
5	0	0	1	0	1	0	5	0	5
6	0	0	1	1	0	0	6	0	6
7	0	0	1	1	1	0	7	0	7
8	0	1	0	0	0	1	0	0	8
9	0	1	0	0	1	1	1	0	9
10	0	1	0	1	0	1	2	0	A
11	0	1	0	1	1	1	3	0	B
12	0	1	1	0	0	1	4	0	C
13	0	1	1	0	1	1	5	0	D
14	0	1	1	1	0	1	6	0	E
15	0	1	1	1	1	1	7	0	F
16	1	0	0	0	0	2	0	1	0
17	1	0	0	0	1	2	1	1	1
18	1	0	0	1	0	2	2	1	2
19	1	0	0	1	1	2	3	1	3
20	1	0	1	0	0	2	4	1	4

Calculator in PC

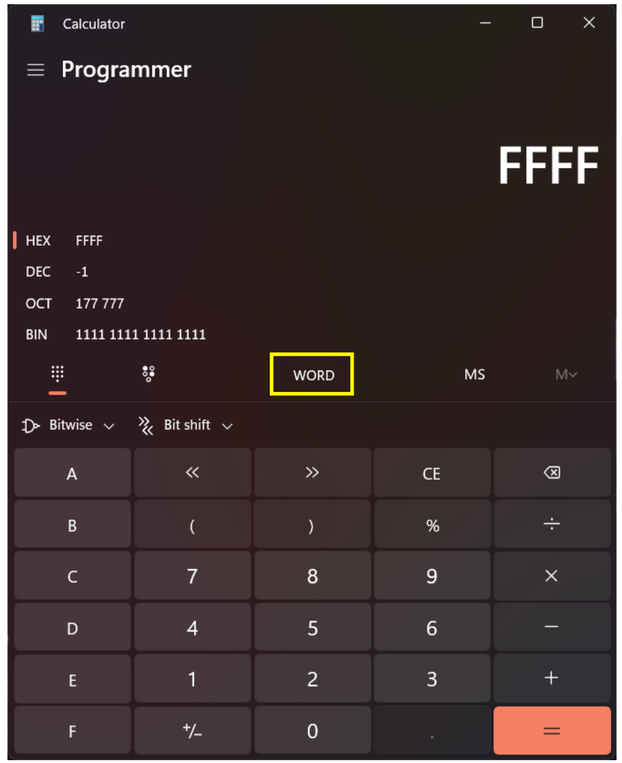
- Windows內建小算盤可以協助你進行2、8、10、和16進制的換算



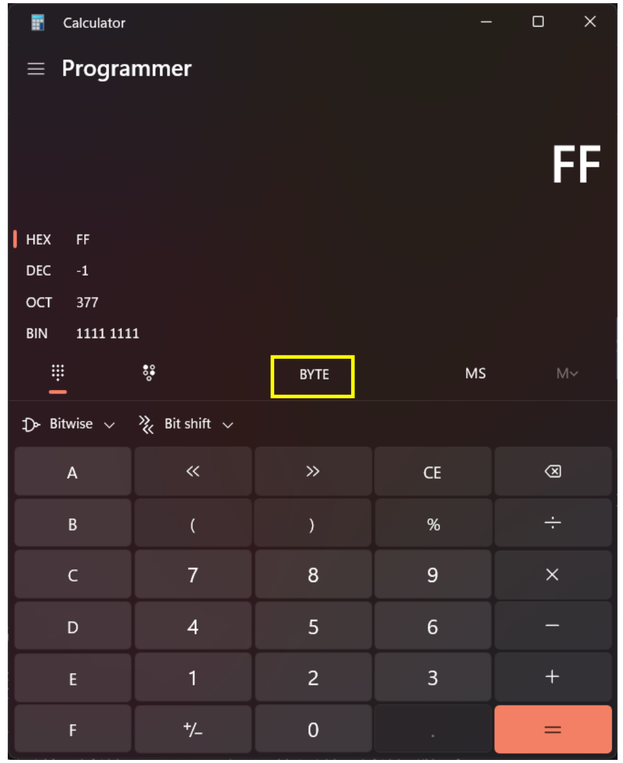
QWORD:8 Bytes(64-bit)



QWORD:4 Bytes(32-bit)



QWORD:2 Bytes(16-bit)



QWORD:1 Byte(8-bit)

Data Type for C

Data Type	Length (bit)	Range
bool	1/8	0(false)/1(true)
(signed) char	8	-128 ~ 127
int8_t	8	
unsigned char	8	0 ~ 255
uint8_t	8	
(signed) short int	16	-32768 ~ 32767
int16_t		
unsigned short int	16	0 ~ 65535
uint16_t		
(signed) int	32/16*	-2147483648 ~ 2147483647/ -32768 ~ 32767*
(signed) long int	32	-2147483648 ~ 2147483647
int32_t		
unsigned int	32/16*	0 ~ 4294967295/ 0 ~ 65535*
unsigned long int	32	0 ~ 4294967295
uint32_t	32	
(signed) long long int	64	-9223372036854775808 ~ 9223372036854775807
int64_t	64	
unsigned long long int	64	0 ~ 18446744073709551615
uint64_t	64	
flaot	32	$2.939 \times 10^{-38} \sim 3.403 \times 10^{+38}$ (7 sf)
double	64	$5.563 \times 10^{-309} \sim 1.798 \times 10^{+308}$ (15 sf)
long double	128	$7.065 \times 10^{-9865} \sim 1.415 \times 10^{+9864}$ (18 sf/33 sf)

*Some compliers define that the data length of the int data type is 16-bit.

Unsigned Representation

Data Type	Length (bit)	Range
bool	1/8	0(false)/1(true)
(signed) char	8	-128 ~ 127
int8_t	8	
unsigned char	8	0 ~ 255
uint8_t	8	
(signed) short int	16	-32768 ~ 32767
int16_t		
unsigned short int	16	0 ~ 65535
uint16_t		
(signed) int	32/16*	-2147483648 ~ 2147483647/ -32768 ~ 32767*
(signed) long int	32	-2147483648 ~ 2147483647
int32_t		
unsigned int	32/16*	0 ~ 4294967295/ 0 ~ 65535*
unsigned long int	32	0 ~ 4294967295
uint32_t	32	
(signed) long long int	64	-9223372036854775808 ~ 9223372036854775807
int64_t	64	
unsigned long long int	64	0 ~ 18446744073709551615
uint64_t	64	
flaot	32	2.939×10^{-38} ~ $3.403 \times 10^{+38}$ (7 sf)
double	64	5.563×10^{-309} ~ $1.798 \times 10^{+308}$ (15 sf)
long double	128	7.065×10^{-9865} ~ $1.415 \times 10^{+9864}$ (18 sf/33 sf)

unsigned char (8-bit)	Binary Number							
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1
2	0	0	0	0	0	0	1	0
3	0	0	0	0	0	0	1	1
4	0	0	0	0	0	1	0	0
5	0	0	0	0	0	1	0	1
⋮	.....							
254	1	1	1	1	1	1	1	0
255	1	1	1	1	1	1	1	1

An “unsigned char” can represent numbers from 0 to 2^8 - 1(0～255)

Signed Representation(Two's complement)

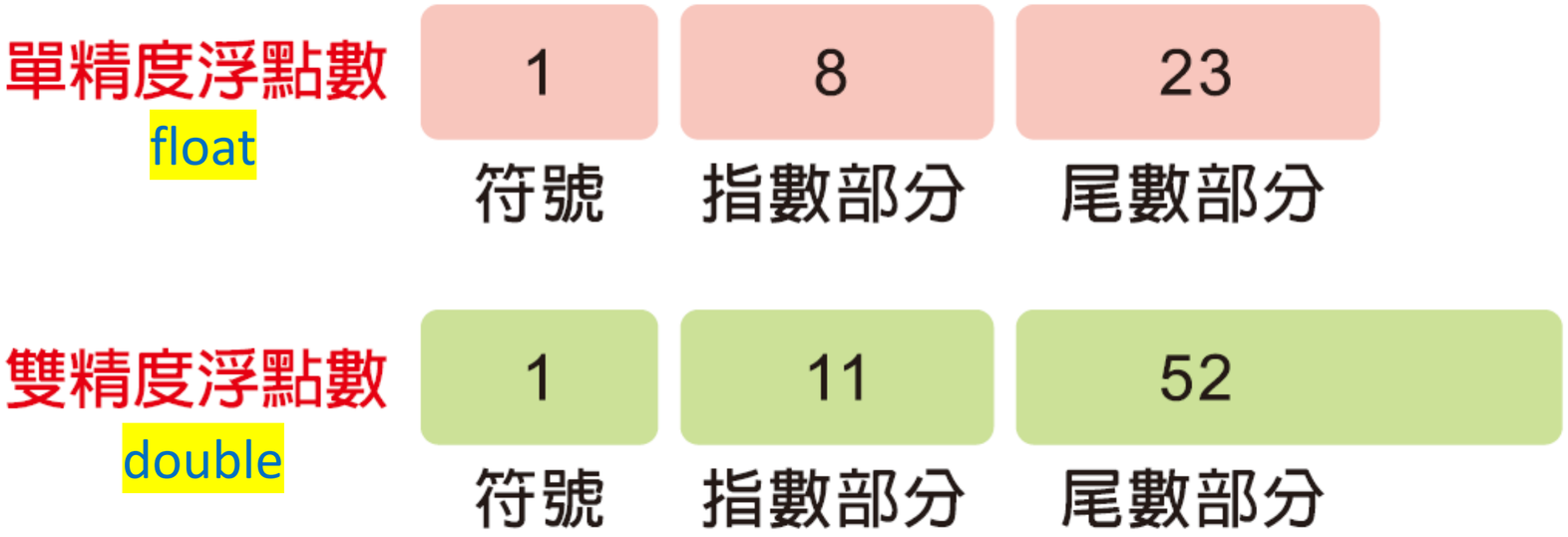
Data Type	Length (bit)	Range
bool	1/8	0(false)/1(true)
(signed) char	8	-128 ~ 127
int8_t	8	
unsigned char	8	0 ~ 255
uint8_t	8	
(signed) short int	16	-32768 ~ 32767
int16_t		
unsigned short int	16	0 ~ 65535
uint16_t		
(signed) int	32/16*	-2147483648 ~ 2147483647/ -32768 ~ 32767*
(signed) long int	32	-2147483648 ~ 2147483647
int32_t		
unsigned int	32/16*	0 ~ 4294967295/ 0 ~ 65535*
unsigned long int	32	0 ~ 4294967295
uint32_t	32	
(signed) long long int	64	-9223372036854775808 ~ 9223372036854775807
int64_t	64	
unsigned long long int	64	0 ~ 18446744073709551615
uint64_t	64	
flaot	32	$2.939x10^{-38}$ ~ $3.403x10^{+38}$ (7 sf)
double	64	$5.563x10^{-309}$ ~ $1.798x10^{+308}$ (15 sf)
long double	128	$7.065x10^{-9865}$ ~ $1.415x10^{+9864}$ (18 sf/33 sf)

Decimals (8-bit)	Binary Number(Sign-magnitude)							
	Bit 7 (sign-bit)	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
127	0	1	1	1	1	1	1	1
126	0	1	1	1	1	1	1	0
125	0	1	1	1	1	1	0	1
⋮	0	.....						
3	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	1
0	0	0	0	0	0	0	0	0
-1	1	1	1	1	1	1	1	1
-2	1	1	1	1	1	1	1	0
-3	1	1	1	1	1	1	0	1
⋮	1	.....						
-126	1	0	0	0	0	0	1	0
-127	1	0	0	0	0	0	0	1
-128	1	0	0	0	0	0	0	0

A "signed char" can represent numbers from to -2^7~2^7-1

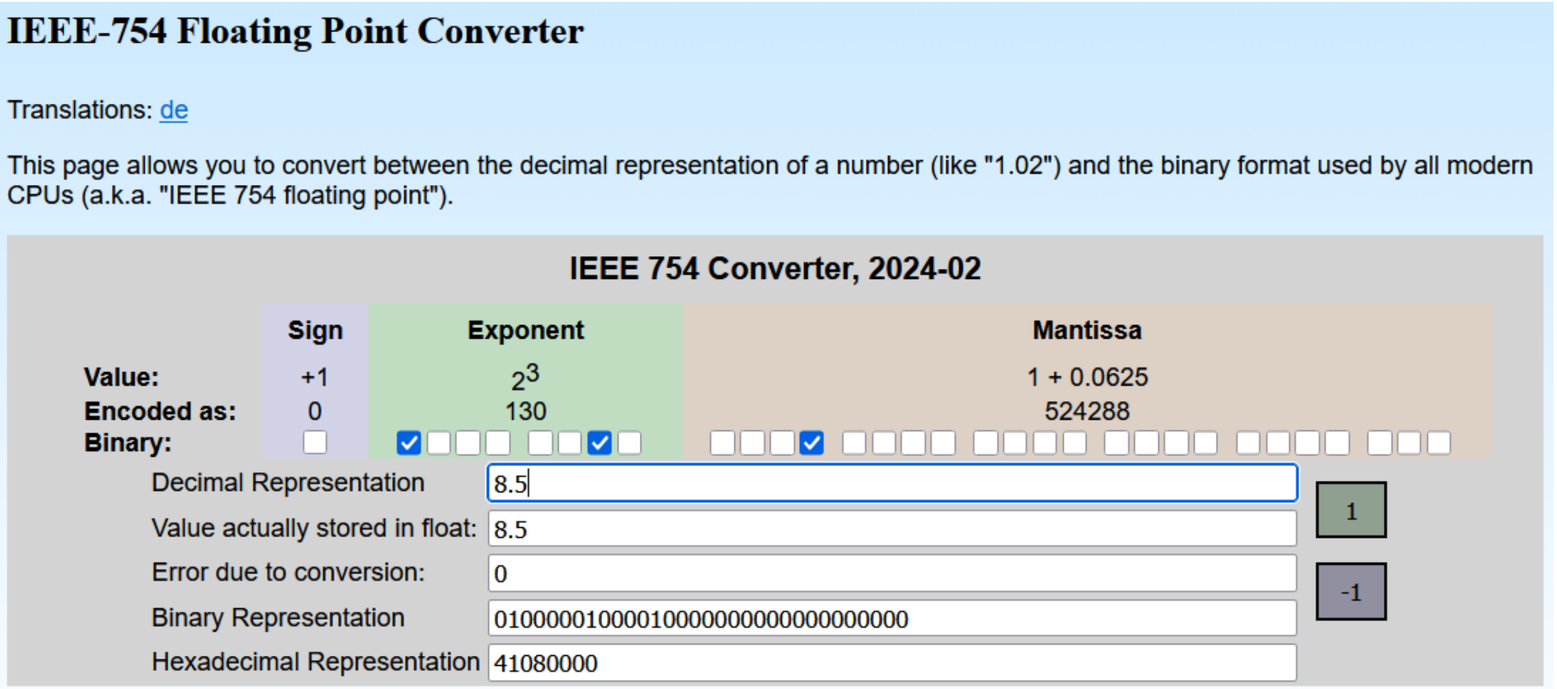
Floating-point representation

- Single-precision floating-point number ：1 bit for sign; 8 bits for exponent; 23 bits for mantissa.
- Double-precision floating-point number ： 1 bit for sign; 11 bits for exponent; 52 bits for mantissa.



Single-Precision Floating-Point Number
單精度浮點數

- Sign bit(符號位元)：1 bit, where 0 represents positive and 1 represents negative.
- Exponent part(指數偏移值) ：8 bits, using the Excess 127 representation (subtracting 127 from the binary value to obtain the actual stored value).
- Mantissa part(尾數部份) ：23 bits, starting from the normalized decimal point, with trailing zeros added if necessary.



https://www.h-schmidt.net/FloatConverter/IEEE754.html

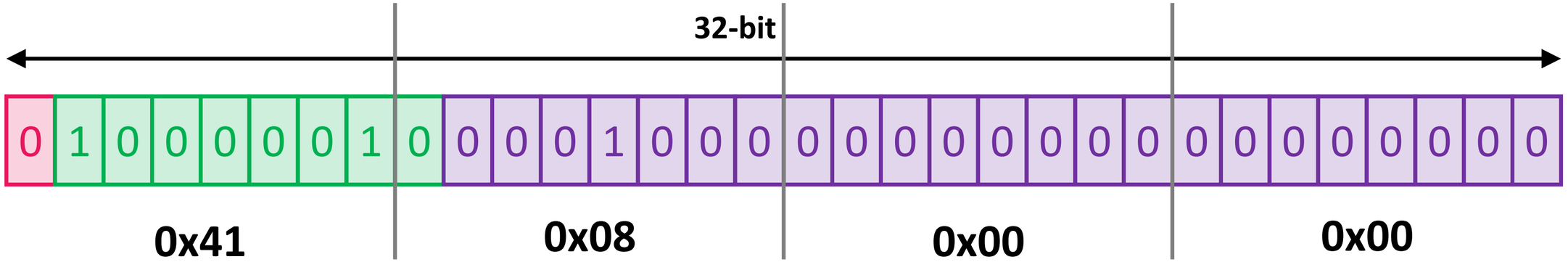
Example – 8.5

STEP 1: 8.5 = $(1000.1)_2$ = $(1.0001x2^3)_2$

STEP 2: Since it is a positive number, the sign bit is 0.

STEP 3: By using the Excess 127 representation, the exponent part is 127+3=130=$(10000010)_2$

STEP 4: The mantissa part is 00010000000000000000000



Single Precision
IEEE 754 Floating-Point Standard

The End